

HTML-Formulare

GET und POST

10 kleine Übungen aus dem Alltag | Startdateien mit TODOs

Ziel

Ein bis zwei Formularfelder pro Aufgabe. Du übst den gleichen Ablauf in unterschiedlichen Situationen: Daten empfangen, auf dem Server prüfen, bei Fehlern zurückleiten und gültige Eingaben wieder einsetzen. Keine Sessions, keine Datenbank, kein JavaScript.

Drei Dateien pro Aufgabe

index.php: HTML-Formular, Fehlermeldung und zurückgegebene Werte.

auswertung.php: Eingaben empfangen, prüfen und darauf reagieren.

function.php: Die Ausgabefunktion **html()** ist vorgegeben. In Aufgabe 10 kommt die Rückleitungsfunktion hinzu.

Die Formularseite heißt index.php, damit PHP die Rückgabewerte direkt in das HTML einsetzen kann. Das Formular selbst bleibt ein HTML-Formular. [1]

Für jede Aufgabe verbindlich

1. Jeden Formulareintrag vor dem Auslesen ausdrücklich mit **isset()** prüfen. Nur Textwerte übernehmen; bei freien Eingaben äußere Leerzeichen entfernen. Danach erst die Inhaltsprüfung. [2]
2. Bei zwei Feldern **immer beide prüfen**, dann zurückleiten. Alle Fehler sammeln und davon getrennt nur gültige Werte übernehmen.
3. Bei Fehlern: mit **header()** zu index.php zurück, danach **exit**. Vorher keine Ausgabe. Die Rückleitung verwendet HTTP 303. [3]
4. Im Formular: Meldung zeigen, **gültige Eingaben erhalten, ungültige leeren**. Das gilt später auch für Auswahl und Checkbox. Der erste Aufruf bleibt leer und ohne Fehler.
5. Die vorbereiteten Formulare nutzen **novalidate**, damit die PHP-Prüfung sichtbar wird. Teste leere und fehlende Werte, reine Leerzeichen und bei zwei Feldern beide Fehler gleichzeitig. [6]

Rückweg und Übungsdaten

Auch nach POST kommen Meldungen und gültige Rückgabewerte hier über die URL zu index.php, also über \$_GET. Es ist ein neuer Aufruf. Nutze nur kurze, erfundene Daten, keine Passwörter oder vertraulichen Angaben. Bei jedem Absenden erneut prüfen. [1, 3]

Start in XAMPP

Startordner unter **C:\xampp\htdocs** entpacken, Apache starten und die jeweilige index.php über **localhost** öffnen, nicht per Doppelklick. Bearbeite die TODOs; die Lösungen liegen getrennt.

01 Büchersuche

GET | Suchbegriff: suchbegriff

Ein Feld empfangen, auf fehlende oder leere Eingaben reagieren.

1. Sende suchbegriff mit GET an auswertung.php. Prüfe den Eintrag in `$_GET` mit `isset()`, bevor du ihn ausliest.
2. Übernimm nur einen Textwert und entferne äußere Leerzeichen. Ein fehlender oder danach leerer Suchbegriff ist ungültig.
3. Leite bei einem Fehler mit `header()` zum Formular zurück. Zeige dort die Fehlermeldung; das Feld bleibt leer.
4. Bei gültiger Eingabe gib aus, wonach gesucht wird. Eine echte Büchersuche wird nicht programmiert.

Teste selbst

Roboter → Ausgabe des Suchbegriffs.

Leer oder nur Leerzeichen → Fehlermeldung im Formular.

auswertung.php ohne Parameter öffnen → Rückleitung, keine PHP-Warnung.

02 Chatname festlegen

POST | Chatname: alias

Den Ablauf auf POST übertragen und die Übertragungsmethode unterscheiden.

1. Sende alias mit POST. Prüfe ausdrücklich mit `isset()`, ob der Eintrag in `$_POST` gesetzt ist; lies nicht aus `$_GET`.
2. Übernimm nur einen Textwert. Entferne äußere Leerzeichen und lehne eine fehlende oder leere Eingabe ab.
3. Leite bei einem Fehler zurück. Die Meldung kommt als URL-Parameter im Formular an und wird dort angezeigt.
4. Bei gültiger Eingabe gib den Chatnamen aus. Ein GET-Aufruf der Auswertung darf nicht als erfolgreiche POST-Eingabe gelten.

Teste selbst

Pixel → Erfolgsausgabe.

Leer oder nur Leerzeichen → Rückleitung mit Fehlermeldung.

auswertung.php?alias=Pixel öffnen → keine erfolgreiche POST-Auswertung.

03 Fundbüro: Gegenstand beschreiben

GET | Gegenstand: gegenstand · Farbe: farbe

Zwei Pflichtfelder getrennt prüfen und gültige Werte wieder einsetzen.

1. Empfange gegenstand und farbe mit GET. Prüfe jeden Eintrag zuerst mit isset(); beide Textfelder sind Pflichtfelder.
2. Entferne äußere Leerzeichen. Prüfe beide Felder unabhängig und sammle alle Fehlermeldungen sowie die gültigen Werte.
3. Leite erst nach beiden Prüfungen zurück. Zeige die Meldungen, setze gültige Werte wieder ein und lasse ungültige Felder leer.
4. Sind beide Werte gültig, gib Gegenstand und Farbe aus.

Teste selbst

Schirm / leer → Schirm bleibt erhalten, Farbe bleibt leer.

Leer / blau → blau bleibt erhalten; beide leer → zwei Feldmeldungen.

Schirm / blau → Erfolg; lösche danach einen Parameter aus der GET-URL.

04 Kontakt: Name und E-Mail

POST | Name: name · E-Mail-Adresse: email

Zusätzlich zum Pflichtfeld auch das Format einer Eingabe prüfen.

1. Empfange name und email mit POST. Prüfe beide Einträge mit isset(), übernimm nur Textwerte und entferne äußere Leerzeichen.
2. Der Name darf nicht leer sein. Die E-Mail-Adresse muss zusätzlich die PHP-Prüfung mit FILTER_VALIDATE_EMAIL bestehen.
3. Prüfe beide Felder vor der Rückleitung. Erhalte nur gültige Angaben; eine falsche E-Mail-Adresse wird geleert.
4. Bei Erfolg gib Name und E-Mail-Adresse aus. Es wird keine E-Mail verschickt und kein Postfach auf Existenz geprüft.

Teste selbst

Mia / mia@example.org → Erfolg.

Mia / keine-mail → Name bleibt, E-Mail-Feld leer.

Leer / mia@example.org → Adresse bleibt; beide leer → beide Meldungen.

05 Kino: Kartenwunsch prüfen

GET | Film: film · Anzahl Karten: karten (1 bis 6, ganz)

Eine ganze Zahl und einen erlaubten Zahlenbereich kontrollieren.

1. Empfange film und karten mit GET. Prüfe beide Einträge mit `isset()`, übernimm Textwerte und entferne äußere Leerzeichen.
2. Der Filmtitel ist Pflicht. Die Kartenanzahl muss aus Ziffern bestehen und als ganze Zahl zwischen 1 und 6 liegen.
3. Prüfe die Zahl in PHP, nicht nur im Browser. Leite bei Fehlern zurück und erhalte jeweils das gültige Feld.
4. Sind beide Werte gültig, gib Filmtitel und Kartenanzahl aus. Es wird nichts reserviert.

Teste selbst

Abenteuer / 2 → Erfolg.

Abenteuer / 0, 7, 2.5 oder abc → Film bleibt, Kartenanzahl leer.

Leer / 2 → Kartenanzahl bleibt; fehlende Anzahl → Fehlermeldung.

06 Lagerbestand: Null ist erlaubt

POST | Artikel: artikel · Bestand: bestand (0 bis 50, ganz)

Einen fehlenden Wert von der ausdrücklich gültigen Zahl 0 unterscheiden.

1. Empfange artikel und bestand mit POST. Prüfe beide Einträge mit `isset()`, übernimm nur Textwerte und entferne äußere Leerzeichen.
2. Der Artikel ist Pflicht. Erlaubt sind ganze Bestandszahlen von 0 bis 50; fehlend oder leer ist nicht dasselbe wie 0.
3. Leite bei Fehlern zurück und erhalte gültige Werte, ausdrücklich auch die 0. Vermeide eine Prüfung, die 0 pauschal als leer ablehnt.
4. Bei gültigen Daten gib Artikel und Bestand aus. Es wird kein Lagerbestand gespeichert.

Teste selbst

Stift / 0 → Erfolg.

Leer / 0 → Artikel leer, die 0 bleibt erhalten.

Stift / leer, -1 oder 51 → Artikel bleibt, Bestand leer.

07 Kursfilter: Auswahl kontrollieren

GET | Thema: thema · Kursformat: format (online / praesenz)

Nur erlaubte Auswahlwerte akzeptieren und eine gültige Auswahl erhalten.

1. Empfange thema und format mit GET. Prüfe für beide Einträge isset() und den Texttyp. Das Thema darf nach trim() nicht leer sein.
2. Das Format muss genau online oder praesenz lauten. Eine leere Auswahl oder ein fremder Wert ist ungültig.
3. Leite bei Fehlern zurück. Ein gültiges Thema bleibt stehen; ein gültiges Format wird wieder ausgewählt, ein ungültiges zurückgesetzt.
4. Bei gültigen Angaben gib Thema und Kursformat aus. Verlasse dich nicht darauf, dass ein Auswahlfeld nur erlaubte Werte sendet.

Teste selbst

PHP / keine Auswahl → PHP bleibt erhalten.

Leer / Online → Online bleibt ausgewählt.

In der GET-URL format=hybrid setzen → Fehler; PHP / Präsenz → Erfolg.

08 Leihgerät: Empfang bestätigen

POST | Name: name · Checkbox: bestaetigt (Wert: ja)

Eine nicht gesendete Checkbox erkennen und einen gültigen Haken erhalten.

1. Empfange name und bestaetigt mit POST. Prüfe jeden Eintrag vor dem Auslesen mit isset(); der Name ist nach trim() ein Pflichtfeld.
2. Die Bestätigung ist nur gültig, wenn der Checkbox-Eintrag vorhanden ist, ein Textwert ist und genau ja enthält.
3. Prüfe beide Felder vor der Rückleitung. Ein gültiger Name bleibt erhalten; eine gültige Bestätigung bleibt angehakt.
4. Bei vollständig gültigen Angaben gib eine Empfangsbestätigung mit dem Namen aus. Es wird kein Gerät tatsächlich verbucht.

Teste selbst

Mia / kein Haken → Name bleibt, Meldung zur Bestätigung.

Leer / Haken → Name leer, der Haken bleibt gesetzt.

Mia / Haken → Erfolg; ein fremder Checkbox-Wert wird abgelehnt.

09 Pizza: Eine Angabe ist freiwillig

GET | Pizza: pizza · Extra (freiwillig): extra (kaese / pilze oder leer)

Fehlend, leer, gültig und ungültig bei einem optionalen Feld unterscheiden.

1. Empfange pizza und extra mit GET. Prüfe vor jedem Auslesen mit `isset()`, ob der Eintrag vorhanden ist. Pizza ist ein Pflichtfeld.
2. Extra darf fehlen oder nach `trim()` leer sein. Bei einer Eingabe sind nur kaese oder pilze erlaubt; übermittelte Arrays sind ungültig.
3. Prüfe beide Angaben und leite bei Fehlern zurück. Eine leere freiwillige Angabe erzeugt keinen Fehler; gültige Werte bleiben erhalten.
4. Bei Erfolg gib Pizza und Extra aus. Ohne Extra erscheint: Kein Extra. Es wird keine Bestellung abgeschickt.

Teste selbst

Margherita / leer oder fehlendes extra → Erfolg ohne Extra.

Margherita / ananas → Pizza bleibt, Extra leer.

Leer / pilze → pilze bleibt; extra[]=kaese in der URL → Fehler.

10 Druckbereich: Zwei Werte vergleichen

POST | Von Seite: von (1 bis 100, ganz) · Bis Seite: bis (1 bis 100, ganz)

Bekannte Prüfungen verbinden und die Rückleitung als Funktion wiederverwenden.

1. Empfange von und bis mit POST. Prüfe beide Einträge mit `isset()`, den Texttyp und anschließend die Zahlen: jeweils ganz und von 1 bis 100.
2. Erst wenn beide Einzelwerte gültig sind, vergleiche sie. Bis darf nicht kleiner als Von sein. In diesem Fall bleibt Von erhalten, Bis wird geleert.
3. Ergänze `zurueckZumFormular()` in `function.php`: gültige Daten und Fehler entgegennehmen, URL bauen, mit `header()` zurückleiten und beenden.
4. Verwende die Funktion bei Fehlern. Bei Erfolg gib den Bereich und die Seitenzahl aus; beide Randseiten werden mitgezählt. Es wird nichts gedruckt.

Teste selbst

2 / 5 → Bereich gültig, 4 Seiten; 5 / 5 → 1 Seite.

8 / 3 → Von bleibt 8, Bis leer, Meldung zur Reihenfolge.

Leer / 4 → Bis bleibt 4; 0 / 101 → beide Felder leer und beide Meldungen.

Merkhilfe für die Übungen

Die Bausteine zum Nachschlagen, keine fertigen Aufgabenlösungen.

GET, POST und der Rücksprung

Das method-Attribut bestimmt die Übermittlung; das name-Attribut bestimmt den Schlüssel. auswertung.php liest passend aus \$_GET oder \$_POST, nicht aus \$_REQUEST. Beim Rücksprung mit HTTP 303 ruft der Browser index.php per GET auf. [1, 3]

Gesetzt ist nicht gleich gültig

isset() prüft, ob ein Eintrag existiert und nicht null ist. Ein leerer String kann trotzdem gesetzt sein. is_string() verhindert, dass ein übermitteltes Array als Text behandelt wird. trim() entfernt äußere Leerzeichen. Danach prüfst du den Inhalt. [2]

Leer ist nicht gleich Null

empty() behandelt auch den String "0" als leer. Aufgabe 6 muss die 0 jedoch akzeptieren. Für das Fehler-Array ist empty() nützlich: Es zeigt, ob keine Fehler gesammelt wurden. Fehlende Pflichtfelder müssen trotzdem abgefangen werden. [2]

Format und Zahlen prüfen

filter_var() mit FILTER_VALIDATE_EMAIL prüft das Format einer E-Mail-Adresse, nicht die Existenz des Postfachs. ctype_digit() prüft auf einen nicht leeren String aus Ziffern; den erlaubten Zahlenbereich prüfst du zusätzlich. Für Zahlenvergleiche kannst du danach in int umwandeln. [5]

URL und HTML unterscheiden

http_build_query() erzeugt URL-Parameter aus einem Array. Die vorgegebene Funktion html() nutzt htmlspecialchars() für HTML-Text und value-Attribute in Anführungszeichen. Erst für die URL kodieren, beim Anzeigen für HTML maskieren. Beides ersetzt keine Inhaltsprüfung. [4]

Auswahl, Checkbox und freiwillige Angaben

selected markiert eine Option, checked setzt einen Haken. Eine nicht angehakete Checkbox wird normalerweise gar nicht mitgesendet. Prüfe die erlaubten Werte trotzdem auf dem Server. Freiwillig bedeutet: fehlend oder leer ist erlaubt, ein vorhandener Inhalt muss aber zur Vorgabe passen. [6]

So prüfst du deinen Rückweg

Bei zwei Feldern abwechselnd nur eines falsch eingeben: Das andere muss erhalten bleiben. Dann beide falsch eingeben: Beide Meldungen müssen erscheinen. Öffne auswertung.php ohne Daten. Kontrolliere bei GET die URL. Zahlenfelder sind hier bewusst Textfelder, damit auch abc den Server erreicht.

Technische Quellen

[1] PHP: [Formulare](#) · [\\$_GET](#) · [\\$_POST](#)

[2] PHP: [isset](#) · [is_string](#) · [trim](#) · [empty](#)

[3] PHP: [header](#) · [MDN](#): [HTTP 303](#)

[4] PHP: [http_build_query](#) · [htmlspecialchars](#)

[5] PHP: [filter_var](#) · [Validierungsfiler](#) · [ctype_digit](#)

[6] [MDN](#): [Formular / novalidate](#) · [Auswahl](#) · [Checkbox](#)